

Encuentro 21 — Cadenas: métodos de texto

Procesamiento de texto y validación

1. Metadatos de bloque

Campo	Detalle
Encuentro	21 de 36
Unidad	3 — Procesamiento de texto y validación
Eje temático	3 — Procesamiento de texto y validación
Carácter/Objetivo	Conceptual
Estructura	clase
Duración teórica	120 minutos (2 horas reloj)
Concepto nuevo	Cadenas: métodos de texto
Requisitos previos	Unidad 2: funciones con <code>def / return</code> , colecciones (<code>list</code> , <code>dict</code> , <code>set</code>), bloque <code>if __name__ == "__main__":</code>
Uso de celular	No permitido
Organización del trabajo	Grupos de 2 personas, mismas parejas de la U2. Cambio de rol de quien escribe después del ejercicio independiente.

Reparto de tiempos

Momento	Tiempo
Apertura y puente	10 min
Teoría mínima	20 min
Práctica guiada	35 min
Ejercicio independiente	25 min
Extensión y consolidación	20 min
Cierre	10 min
Total	120 min

2. Objetivos de aprendizaje

1. Procesar texto de entrada con `split`, `strip`, `join` y `replace` para extraer datos estructurados.
2. Aplicar f-strings con formato numérico (`:.2f`) para mostrar resultados monetarios.
3. Combinar métodos de cadenas con funciones para parsear líneas de entrada del usuario.
4. Distinguir entre métodos que devuelven una nueva cadena y los que no modifican el original.

3. Teoría mínima (20 min)

Charla rápida: la terminal de la pizzería

Imaginate que trabajás en una pizzería y un cliente llama por teléfono para hacer un pedido. Te dicen todo junto: «dos muzza,

una napo, una de palmito, y una fugazzeta». Tu cabeza tiene que separar eso en pedazos, sacarle los espacios de más, reconocer cada ítem y armar la comanda. Los métodos de cadena de Python hacen exactamente eso: `split` corta el texto como si fueran las comas de una frase, `strip` limpia los bordes, `join` vuelve a unir, y `replace` cambia una parte por otra.

Lo mínimo indispensable

Las cadenas (`str`) son inmutables. Todo método de cadena devuelve una *nueva* cadena, no modifica la original.

Métodos canónicos del curso:

Método	Qué hace	Ejemplo	Resultado
<code>cadena.split()</code>	Corta por espacios (colapsa repetidos)	<code>"a b c".split()</code>	<code>["a", "b", "c"]</code>
<code>cadena.split(sep)</code>	Corta por el separador dado	<code>"a,b,c".split(",")</code>	<code>["a", "b", "c"]</code>
<code>cadena.strip()</code>	Saca espacios de ambos bordes	<code>" hola ".strip()</code>	<code>"hola"</code>
<code>" ".join(lista)</code>	Une una lista con separador	<code>" ".join(["a","b"])</code>	<code>"a b"</code>
<code>cadena.replace(viejo, nuevo)</code>	Reemplaza todas las ocurrencias	<code>"1,2,3".replace(",",".")</code>	<code>"1.2.3"</code>

f-strings con formato numérico:

```
precio = 12.5
print(f"Total: ${precio:.2f}") # Total: $12.50
```

El formato `:.2f` fija dos decimales. Útil para montos, precios, resultados financieros.

Receta canónica para parsear una línea de entrada:

```
linea = input("Ingresá los ingredientes separados por coma: ")
# 1. strip para bordes, 2. split para separar
partes = [p.strip() for p in linea.split(",")]
```

El `for` dentro de la lista recorre cada parte y le aplica `strip` para sacar espacios alrededor.

4. Práctica guiada (35 min)

Escenario: en la pizzería entran pedidos como texto sin formato. Tenemos que parsearlos para extraer cada ítem y mostrar el precio.

Código completo (`pizza_parser.py`):

```

def parse_order(order_text):
    """
    Convierte un texto como '2 muzza, 1 napo' en una lista de tuplas.
    Cada tupla: (cantidad, nombre, precio_unitario).
    """
    items = []
    # Separa por coma, Limpia bordes de cada parte
    parts = [p.strip() for p in order_text.split(",")]
    for part in parts:
        # Cada parte: "2 muzza" o "1 napo"
        tokens = part.split()
        if len(tokens) >= 2:
            quantity = int(tokens[0])
            name = " ".join(tokens[1:]) # nombre compuesto
            # Precio base según variedad
            unit_price = get_base_price(name)
            items.append((quantity, name, unit_price))
    return items

def get_base_price(pizza_name):
    """Devuelve el precio base de una pizza según su nombre."""
    prices = {
        "muza": 1200.0,
        "napo": 1400.0,
        "palmito": 1600.0,
        "fugazzeta": 1500.0,
        "especial": 1800.0,
        "calabresa": 1700.0,
    }
    # .get devuelve "-" si no encuentra la variedad
    return prices.get(pizza_name, 0.0)

def print_order_summary(items):
    """Muestra el resumen del pedido con f-strings formateados."""
    print("\n=== Resumen del pedido ===")
    total = 0.0
    for quantity, name, unit_price in items:
        subtotal = quantity * unit_price
        total += subtotal
        print(f"{quantity}x {name}: ${unit_price:.2f} c/u → ${subtotal:.2f}")
    print(f"Total del pedido: ${total:.2f}")
    print("=" * 30)

if __name__ == "__main__":
    # Entrada: el usuario tipea el pedido como texto
    raw = input("Ingresá el pedido (ej: 2 muzza, 1 napo, 3 palmito): ")
    parsed = parse_order(raw)
    if parsed:
        print_order_summary(parsed)
    else:
        print("No se pudo interpretar el pedido.")

```

Salida esperada:

```

Ingresá el pedido (ej: 2 muzza, 1 napo, 3 palmito): 2 muzza, 1 napo, 3 calabresa

=== Resumen del pedido ===
2x muzza: $1200.00 c/u → $2400.00
1x napo: $1400.00 c/u → $1400.00
3x calabresa: $1700.00 c/u → $5100.00
Total del pedido: $8900.00
=====

```

Pasos para la puesta en común: 1. Escribí el código en un archivo `pizza_parser.py`. 2. Ejecutalo con `python pizza_parser.py`. 3. Probá con distintas entradas: "1 especial, 2 fugazzeta", "3 muzza". 4. ¿Qué pasa si ponés "muzza, napo" sin cantidad? — se rompe porque `int("muzza")` falla. Descubrimos que necesitamos validación (lo veremos en la clase 22).

5. Ejercicio independiente (25 min)

Consigna: escribí un programa `telefono_cleaner.py` que reciba una línea con nombre de comprador, dirección y teléfono separados por punto y coma (;). El programa debe:

1. Separar los tres campos.
2. Limpiar espacios de bordes en cada campo.
3. Si el teléfono tiene espacios o guiones (ej: "11 1234 5678"), debe sacarlos con `replace` para dejar solo dígitos.
4. Mostrar los datos limpios con f-strings.

Ejemplo de entrada:

```

María García; Av. Siempre Viva 123; 11 2345 6789

```

Salida esperada:

```

Nombre: María García
Dirección: Av. Siempre Viva 123
Teléfono: 1123456789

```

Pista: `replace` puede sacar espacios (" ") y también guiones ("-"). Podés encadenar dos `replace`: uno para cada carácter que querés eliminar.

Solución: en el anexo docente. Intentá resolverlo sin mirar la respuesta.

6. Extensión y consolidación (20 min)

Para quienes terminan el ejercicio base:

1. **Agregá formato monetario con separador de miles:** investigá cómo usar `:,.2f` en f-strings para que `1234567.50` se muestre como `1,234,567.50`.
2. **Validación mínima:** si el teléfono limpio no tiene al menos 7 dígitos, mostrá un mensaje de advertencia.
3. **Procesamiento batch:** permití que el usuario ingrese tres líneas de pedido seguidas; procesalas en un `for` y mostrá un resumen final.

7. Cierre (10 min)

Qué te llevás

- Las cadenas son inmutables: los métodos devuelven un nuevo `str`, no cambian el original.
- `split`, `strip`, `join` y `replace` son la caja de herramientas para procesar texto del usuario.
- Las f-strings con `:.2f` dan formato limpio a valores numéricos.
- Combo canónico: `strip` primero, `split` después, procesar cada parte.

Lo que viene

En el **Encuentro 22** vamos a resolver el problema que apareció hoy: ¿qué pasa cuando el usuario tipea texto donde esperamos un número? Vamos a aprender **validación con** `try/except`.

8. Errores comunes y trampas

Error	Causa	Fix
<code>cadena = cadena.split()</code> y después querés usar la original	<code>split()</code> devuelve una nueva lista, la cadena original no se modifica	Guardá el resultado en otra variable o reasígnala
<code>" hola ".strip()</code> y esperar que saque espacios internos	<code>strip</code> solo saca de los bordes, no del medio	Usá <code>replace(" ", "")</code> o <code>split() + join()</code> para espacios internos
<code>linea.split(",")</code> con <code>"a, b, c"</code> deja espacios en <code>" b"</code>	<code>split</code> no saca espacios alrededor del separador	Aplicá <code>strip()</code> a cada parte después de <code>split</code>
<code>f"{precio}"</code> muestra <code>12.5</code> en vez de <code>12.50</code>	Sin formato, Python no agrega ceros	Usá <code>f"{precio:.2f}"</code>
<code>join</code> al revés: <code>lista.join(" ")</code> en vez de <code>" ".join(lista)</code>	<code>join</code> es método del separador, no de la lista	Recordá: <code>separador.join(iterable)</code>
Usar <code>"hola".replace("")</code> para algo	<code>replace</code> con cadena vacía está bien, pero cuidado al borrar todo	Es válido, pero no hace nada útil si buscás otra cosa

Encuentro 22 — Validación con try/except

Procesamiento de texto y validación

1. Metadatos de bloque

Campo	Detalle
Encuentro	22 de 36
Unidad	3 — Procesamiento de texto y validación
Eje temático	3 — Procesamiento de texto y validación
Carácter/Objetivo	Procedimental
Estructura	clase
Duración teórica	120 minutos (2 horas reloj)
Concepto nuevo	Validación con try/except
Requisitos previos	Encuentro 21: métodos de cadenas (<code>split</code> , <code>strip</code> , <code>join</code> , <code>replace</code>) y f-strings con formato
Uso de celular	No permitido
Organización del trabajo	Grupos de 2 personas. Una computadora por grupo, alternan quien escribe en cada ejercicio.

Reparto de tiempos

Momento	Tiempo
Apertura y puente	10 min
Teoría mínima	20 min
Práctica guiada	35 min
Ejercicio independiente	25 min
Extensión y consolidación	20 min
Cierre	10 min
Total	120 min

2. Objetivos de aprendizaje

1. Identificar cuándo `int()` o `float()` lanzan `ValueError` al recibir texto no numérico.
2. Usar `try/except ValueError` para capturar el error y mostrar un mensaje accionable.
3. Implementar el patrón de reintento `while True` con validación.
4. Distinguir entre errores de usuario (entrada inválida) y errores de programa (bug), y decidir cuáles atrapar.

3. Teoría mínima (20 min)

Charla rápida: el portero del boliche

En la puerta de un boliche hay un portero que solo deja entrar a mayores de edad. Vos llegás y decís «diecisiete»; el portero no

te entiende porque necesita un número. Te pide «decime tu edad en número». Eso mismo hace `int()` cuando recibe texto: solo entiende dígitos. Si llegás con `"diecisiete"`, `int()` protesta. `try/except` es el protocolo: intentá convertir, y si falla, atajá el error y pedí de nuevo.

Lo mínimo indispensable

La excepción `ValueError`: cuando llamás `int("hola")`, Python lanza `ValueError` porque el texto no es numérico. Lo mismo con `float()`.

Patrón canónico de validación (único autorizado en el curso):

```
def read_int(message):
    """Pide un entero y reintentá hasta obtener uno válido."""
    while True:
        try:
            return int(input(message))
        except ValueError:
            print("Eso no es un número entero; intentá de nuevo.")
```

Reglas de la casa: - `except:` desnudo está **prohibido**. Siempre nombrá la excepción: `except ValueError`. - El mensaje de error debe ser **accionable**: decirle al usuario qué hacer, no solo «Error». - El bloque `try` debe ser lo más chico posible: solo la conversión, no toda la función. - `while True` con `return` en el `try` es el patrón: se sale solo cuando la conversión es exitosa.

Variante con `float`:

```
def read_float(message):
    while True:
        try:
            return float(input(message))
        except ValueError:
            print("Eso no es un número válido (ej: 12.5); intentá de nuevo.")
```

Qué NO se atrapa con `try/except`: - Una opción inválida de menú (se resuelve con `if/else` y mensaje). - Una clave que falta en un diccionario (se resuelve con `.get()` o `in`). - Un error de lógica del programa (eso es un bug, no se tapa).

4. Práctica guiada (35 min)

Escenario: en la pizzería necesitamos registrar pedidos con cantidad y precio. El usuario puede equivocarse al tipear. Vamos a hacer una función de carga validada.

Código completo (`order_input.py`):

```

def read_int(message):
    """Solicita un entero con validación y reintento."""
    while True:
        try:
            return int(input(message))
        except ValueError:
            print("❌ Eso no es un número entero; escribí solo dígitos (ej: 3).")

def read_float(message):
    """Solicita un número decimal con validación y reintento."""
    while True:
        try:
            return float(input(message))
        except ValueError:
            print("❌ Eso no es un número válido; usá punto decimal (ej: 12.50).")

def add_pizza_to_order(order_list):
    """Agrega una pizza al pedido pidiendo cantidad y precio."""
    print("\n--- Nueva pizza ---")
    name = input("Nombre de la pizza: ").strip()
    quantity = read_int(f"Cantidad de {name}: ")
    unit_price = read_float(f"Precio unitario de {name}: $")

    # Calcula subtotal
    subtotal = quantity * unit_price
    order_list.append((name, quantity, unit_price, subtotal))
    print(f"✓ Agregadas {quantity}x {name} a ${unit_price:.2f} = ${subtotal:.2f}")

def show_order(order_list):
    """Muestra el pedido completo."""
    if not order_list:
        print("El pedido está vacío.")
        return

    print("\n=== Pedido completo ===")
    total = 0.0
    for name, qty, price, sub in order_list:
        total += sub
        print(f"{qty}x {name}: ${price:.2f} c/u → ${sub:.2f}")
    print(f"Total: ${total:.2f}")

if __name__ == "__main__":
    order = []
    print("=== Registro de pedidos de la pizzería ===")
    add_pizza_to_order(order)
    add_pizza_to_order(order)
    show_order(order)

```

Salida esperada (con errores incluidos):

```

=== Registro de pedidos de la pizzería ===

--- Nueva pizza ---
Nombre de la pizza: muzza
Cantidad de muzza: dos
❑ Eso no es un número entero; escribí solo dígitos (ej: 3).
Cantidad de muzza: 2
Precio unitario de muzza: $doce
❑ Eso no es un número válido; usá punto decimal (ej: 12.50).
Precio unitario de muzza: $1200
✓ Agregadas 2x muzza a $1200.00 = $2400.00

--- Nueva pizza ---
Nombre de la pizza: napo
Cantidad de napo: 1
Precio unitario de napo: $1400
✓ Agregadas 1x napo a $1400.00 = $1400.00

=== Pedido completo ===
2x muzza: $1200.00 c/u → $2400.00
1x napo: $1400.00 c/u → $1400.00
Total: $3800.00

```

Pasos para la puesta en común: 1. Escribí el código en `order_input.py`. 2. Ejecutalo y probá la ruta feliz (números válidos). 3. Reiniciá y probá escribir texto donde va un número. 4. Discutí: ¿qué pasa si en vez de `except ValueError` escribimos `except: ?` — el docente explica por qué está prohibido.

5. Ejercicio independiente (25 min)

Consigna: escribí un programa `read_age.py` que pida la edad del comprador (entre 0 y 120) con validación. Usá el patrón `read_int` visto, pero agregá una segunda validación: si la edad no está entre 0 y 120, mostrá un mensaje y volvé a pedir.

Formato esperado:

```

Ingresá tu edad: -5
❑ La edad debe estar entre 0 y 120 años; intentá de nuevo.
Ingresá tu edad: 150
❑ La edad debe estar entre 0 y 120 años; intentá de nuevo.
Ingresá tu edad: 25
✓ Edad registrada: 25 años.

```

Pista: después del `return int(...)`, antes de salir de la función, chequeá con `if` si el valor está en el rango permitido. Si no, mostrá el mensaje y el `while` se encarga de pedir de nuevo.

Solución: en el anexo docente.

6. Extensión y consolidación (20 min)

Para quienes terminan el ejercicio base:

- Agregá validación de rango superior:** en la pizzería, la cantidad máxima por pizza es 99. Modificá `read_int` para que acepte un mínimo y un máximo opcionales.
- Menú simple con opción numérica:** hacé un programa que muestre 3 opciones de pizzas numeradas y pida elegir. Si el

usuario tipea 4, 0 o letras, debe rechazar y volver a pedir.

3. **Múltiples intentos acotados:** modificá el patrón para que después de 3 intentos fallidos el programa termine con un mensaje.

7. Cierre (10 min)

Qué te llevás

- `int()` y `float()` lanzan `ValueError` cuando reciben texto no numérico — es esperado, no es un bug del programa.
- El patrón `while True` + `try/except ValueError` + `return` es la receta canónica para validar entrada.
- `except:` desnudo está prohibido: siempre nombrá la excepción que esperás.
- Los errores de validación son del usuario; los bugs son del programador. No se mezclan.

Lo que viene

En el **Encuentro 23** vamos a meter todo esto adentro de un **menú de consola en memoria**, con `import` de módulos built-in como `random` y `math`.

8. Errores comunes y trampas

Error	Causa	Fix
<code>except:</code> sin nombre de excepción	El programador tapa todos los errores, incluso bugs	Siempre nombrar <code>except ValueError:</code>
Poner todo el programa dentro de <code>try</code>	Atrapa errores que no deberían atraparse	Poné solo la conversión dentro del <code>try</code>
Validar el rango pero olvidar el <code>while</code>	Si la edad está fuera de rango, no vuelve a pedir	Validá dentro del mismo <code>while True</code> que pide el número
<code>int(input(...))</code> fuera del <code>try</code>	<code>ValueError</code> sin atrapar corta el programa	Encerrá la línea completa <code>int(input(...))</code> en el <code>try</code>
Mensaje de error genérico: <code>"Error: entrada inválida"</code>	No ayuda al usuario a corregir	El mensaje debe decir qué espera el programa y cómo corregirlo
Usar <code>try/except</code> para opción de menú inválida	No es un <code>ValueError</code> , es una opción que no existe	Usá <code>if/else</code> con mensaje "Opción inválida"
Validar el rango antes de devolver y olvidar el <code>return</code>	La función devuelve <code>None</code> si no hay <code>return</code>	Asegurate de que <code>return</code> esté después de validar

Encuentro 23 — Módulos y menú en memoria

Procesamiento de texto y validación

1. Metadatos de bloque

Campo	Detalle
Encuentro	23 de 36
Unidad	3 — Procesamiento de texto y validación
Eje temático	3 — Procesamiento de texto y validación
Carácter/Objetivo	Procedimental
Estructura	clase
Duración teórica	120 minutos (2 horas reloj)
Concepto nuevo	Módulos y menú en memoria
Requisitos previos	Encuentro 22: validación con <code>try/except ValueError</code> y patrón de reintento
Uso de celular	No permitido
Organización del trabajo	Grupos de 2 personas. En este encuentro cada grupo arma su primer menú funcional completo.

Reparto de tiempos

Momento	Tiempo
Apertura y puente	10 min
Teoría mínima	20 min
Práctica guiada	35 min
Ejercicio independiente	25 min
Extensión y consolidación	20 min
Cierre	10 min
Total	120 min

2. Objetivos de aprendizaje

1. Usar `import` para incorporar funciones de módulos built-in (`random.randint`, `math.sqrt`).
2. Diseñar un menú de consola con opciones numeradas y un bucle principal `while True`.
3. Integrar funciones, colecciones, validación y procesamiento de texto en un solo programa.
4. Distinguir entre módulos built-in (que vienen con Python) y módulos propios (que no usamos en este curso).

3. Teoría mínima (20 min)

Charla rápida: la caja de herramientas de la pizzería

La pizzería tiene una caja de herramientas compartida: un horno (que todos usan), una calculadora de precios, un generador de

números de ticket. En Python, esas herramientas compartidas se llaman **módulos**. `import` es la forma de pedir: «che, préstame el `random` para generar un número de ticket al azar» o « préstame `math` para calcular el área de la pizza». No los escribimos nosotros: ya vienen con Python.

Lo mínimo indispensable

Los módulos son archivos de Python con funciones ya escritas. Se importan con `import` al principio del archivo:

```
import random
import math

# Uso:
num = random.randint(1, 100)      # entero aleatorio entre 1 y 100
raiz = math.sqrt(144)             # raíz cuadrada: 12.0
```

Reglas del curso sobre import: - Solo `import` de módulos **built-in** (los que vienen instalados con Python). No vamos a instalar paquetes externos. - `import` va al **principio del archivo**, después de los comentarios, antes de las funciones. - Usamos la sintaxis `import modulo` (no `from modulo import *`). - Para llamar una función del módulo: `random.randint(...)` o `math.sqrt(...)`.

Menú de consola en memoria — patrón canónico:

```
import random

def show_menu():
    print("\n=== Pizzería El Pythonista ===")
    print("1. Agregar pizza al pedido")
    print("2. Ver pedido actual")
    print("3. Generar número de ticket")
    print("0. Salir")
    return input("Elegí una opción: ")

if __name__ == "__main__":
    order = [] # memoria: el estado vive acá
    while True:
        option = show_menu()
        if option == "0":
            print("¡Gracias por tu pedido!")
            break
        elif option == "1":
            # agregar pizza (función)
            pass # placeholder: lo completamos en práctica guiada
        elif option == "2":
            # ver pedido
            pass
        elif option == "3":
            ticket = random.randint(1000, 9999)
            print(f"🎫 Número de ticket: #{ticket}")
        else:
            print("🚫 Opción inválida; elegí 0-3.")
```

4. Práctica guiada (35 min)

Escenario: pizzería con menú completo en memoria. Integramos todo lo visto hasta ahora: funciones que procesan texto, validan entrada con `try/except`, y usan `random` para el ticket.

Código completo (`pizza_menu.py`):

```
import random

def read_int(message, min_val=None, max_val=None):
    """Pide un entero con validación y rango opcional."""
    while True:
        try:
            value = int(input(message))
            if min_val is not None and value < min_val:
                print(f"❌ El mínimo es {min_val}; intentá de nuevo.")
                continue
            if max_val is not None and value > max_val:
                print(f"❌ El máximo es {max_val}; intentá de nuevo.")
                continue
            return value
        except ValueError:
            print("❌ Eso no es un número entero; intentá de nuevo.")

def parse_pizza_line(line):
    """Procesa una línea como '2 muzza' y devuelve (cant, nombre)."""
    parts = line.strip().split()
    if len(parts) < 2:
        return None
    try:
        quantity = int(parts[0])
        name = " ".join(parts[1:])
        return (quantity, name)
    except ValueError:
        return None

def add_pizza(order_list):
    """Agrega una pizza al pedido interactivamente."""
    print("\n--- Agregar pizza ---")
    line = input("Ingresá cantidad y nombre (ej: 2 muzza): ")
    parsed = parse_pizza_line(line)
    if parsed is None:
        print("❌ Formato incorrecto; usá: cantidad nombre")
        return
    quantity, name = parsed
    # Precio fijo por simplicidad (lo podría calcular con dict)
    unit_price = 1200.0 if name.lower() == "muza" else 1400.0
    subtotal = quantity * unit_price
    order_list.append((name, quantity, unit_price, subtotal))
    print(f"✓ Agregadas {quantity}x {name} a ${unit_price:.2f} = ${subtotal:.2f}")

def show_order(order_list):
    """Muestra el pedido actual."""
```

```

if not order_list:
    print("\n El pedido está vacío.")
    return
print("\n=== Pedido actual ===")
total = 0.0
for name, qty, price, sub in order_list:
    total += sub
    print(f"{qty}x {name}: ${price:.2f} c/u → ${sub:.2f}")
print(f"Total: ${total:.2f}")

def show_menu():
    """Muestra el menú y devuelve la opción elegida."""
    print("\n" + "=" * 35)
    print("  Pizzería El Pythonista")
    print("=" * 35)
    print("1. Agregar pizza")
    print("2. Ver pedido")
    print("3. Generar ticket")
    print("0. Salir")
    return input("Elegí una opción: ").strip()

if __name__ == "__main__":
    order = []
    print("¡Bienvenido a la Pizzería El Pythonista!")

    while True:
        option = show_menu()

        if option == "0":
            print(" ¡Gracias por tu pedido! Que lo disfrutes.")
            break
        elif option == "1":
            add_pizza(order)
        elif option == "2":
            show_order(order)
        elif option == "3":
            ticket = random.randint(100, 999)
            print(f" Ticket #{ticket} – guardá este número.")
        else:
            print(" Opción inválida. Elegí 0, 1, 2 o 3.")

```

Salida esperada (secuencia de prueba):

```
¡Bienvenido a la Pizzería El Pythonista!

=====
  🍕 Pizzería El Pythonista
=====

1. Agregar pizza
2. Ver pedido
3. Generar ticket
0. Salir
Elegí una opción: 1

--- Agregar pizza ---
Ingresá cantidad y nombre (ej: 2 muzza): 2 muzza
✓ Agregadas 2x muzza a $1200.00 = $2400.00

Elegí una opción: 1
--- Agregar pizza ---
Ingresá cantidad y nombre (ej: 2 muzza): 1 napo
✓ Agregadas 1x napo a $1400.00 = $1400.00

Elegí una opción: 2

=== Pedido actual ===
2x muzza: $1200.00 c/u → $2400.00
1x napo: $1400.00 c/u → $1400.00
Total: $3800.00

Elegí una opción: 3
🍕 Ticket #742 – guardá este número.

Elegí una opción: 0
🍕 ¡Gracias por tu pedido! Que lo disfrutes.
```

Pasos: 1. Escribí el archivo completo `pizza_menu.py` tal como está arriba. 2. Ejecutalo y probá cada opción del menú. 3. Probá opciones inválidas (letra `x`, número `9`). 4. Probá el parseo con formato incorrecto (ej: `muzza` sin cantidad).

5. Ejercicio independiente (25 min)

Consigna: partiendo de `pizza_menu.py`, agregá una opción **4. Ver ticket activo** que recuerde el último ticket generado y lo muestre de nuevo. Si todavía no se generó ningún ticket, mostrará “Todavía no hay ticket; generá uno con la opción 3.”

Además, modificá la opción 2 para que muestre también la **cantidad total de pizzas** (no solo el total en dinero) al final.

Pista: necesitás una variable global (bueno, una variable en el bloque `__main__`) para guardar el último ticket. Inicializala en `None` y actualizala cuando generás uno nuevo.

Solución: en el anexo docente.

6. Extensión y consolidación (20 min)

Para quienes terminan el ejercicio base:

1. **Agregá `math.sqrt`**: cuando el pedido supera los \$5000, mostrará un mensaje que diga “¡Pedido grande! La raíz cuadrada del total es {total_raiz:.2f}” usando `math.sqrt`.

2. **Precio dinámico con random:** cuando se agrega una pizza, que el precio unitario varíe un poco: `random.randint(1100, 1500)`.
3. **Confirmación antes de salir:** cuando el usuario elige 0, preguntá “¿Estás seguro? (s/n)” antes de cortar.

7. Cierre (10 min)

Qué te llevás

- `import` es la forma de usar herramientas que ya vienen con Python: `random`, `math`.
- El menú de consola vive en un `while True` que solo corta con `break`.
- Las opciones del menú se resuelven con `if/elif/else`, no con `try/except`.
- Integrar funciones + colecciones + validación + texto en un solo programa es el núcleo de la U3.

Lo que viene

Encuentro 24: Integración: menú del TP

8. Errores comunes y trampas

Error	Causa	Fix
Escribe <code>from random import *</code>	Quiere importar todo sin prefijo	Usar <code>import random</code> y llamar <code>random.randint(...)</code>
Olvida el <code>import</code> al principio	El módulo no está disponible	<code>import</code> debe ir antes de cualquier uso del módulo
Escribe <code>import math</code> después de usarlo	Python ejecuta en orden; el módulo no está cargado	Mover todos los <code>import</code> al principio del archivo
Menú eterno sin opción de salida	Falta <code>option == "0"</code> con <code>break</code>	Siempre incluir una opción de salida en el menú
Usa <code>try/except</code> para la opción inválida	Confunde validación de entrada con control de menú	La opción inválida se resuelve con <code>else</code> y un mensaje simple
Llama <code>randint()</code> sin el prefijo <code>random.</code>	Importó el módulo pero no usa el nombre completo	Llamar <code>random.randint()</code> con el nombre del módulo adelante
La variable <code>order</code> está fuera del <code>while</code> pero el <code>while</code> la modifica	Funciona porque <code>order</code> es una lista mutable; confunde el alcance	Explicar que las listas son mutables y se modifican in-place
<code>import</code> con espacios o tildes en el path del archivo	El módulo <code>random</code> está instalado en Python, no en el directorio	No es un path de archivo: es el nombre del módulo en la biblioteca estándar

Encuentro 24 — Integración: menú del TP

Procesamiento de texto y validación

1. Metadatos de bloque

Campo	Detalle
Encuentro	24 de 36
Unidad	3 — Procesamiento de texto y validación
Eje temático	3 — Procesamiento de texto y validación
Carácter/Objetivo	Procedimental
Estructura	clase
Duración teórica	120 minutos (2 horas reloj)
Concepto nuevo	Integración: menú del TP
Requisitos previos	Encuentro 23: menú de consola en memoria con <code>import</code> , funciones y validación
Uso de celular	No permitido
Organización del trabajo	Grupos de 2 personas. Hoy arrancamos el TP-U3: cada grupo arma su propio programa.

Reparto de tiempos

Momento	Tiempo
Apertura y puente	10 min
Teoría mínima	20 min
Práctica guiada	35 min
Ejercicio independiente	25 min
Extensión y consolidación	20 min
Cierre	10 min
Total	120 min

2. Objetivos de aprendizaje

1. Diseñar un programa completo en un solo archivo `.py` que integre funciones, colecciones, validación y procesamiento de texto.
2. Implementar un menú de pizzería con registro de pedidos, cálculo de totales y consultas.
3. Aplicar el patrón `try/except` para validar cada entrada del usuario.
4. Organizar el código con funciones arriba y ejecución en `if __name__ == "__main__":` como dicta la convención del curso.

3. Teoría mínima (20 min)

Charla rápida: el cuaderno de pedidos

Imaginate que la pizzería tiene un cuaderno donde anota los pedidos. Cada pedido tiene: nombre del comprador, teléfono, lista de pizzas y total. El menú de la clase 23 era un ejemplo simple. Ahora vamos a hacer un programa que reemplace ese cuaderno: registrar pedidos, consultarlos y calcular todo automáticamente. Es el mismo esqueleto, pero más completo.

Lo mínimo indispensable

Estructura del TP-U3 — lo que hay que incluir:

```
tp-u3/
  pedidos.py      ← un solo archivo
```

Funciones mínimas del TP-U3 (dominio pizzería):

Función	Qué hace	¿Usa validación?
<code>read_int(msg, min, max)</code>	Pide entero validado	Sí — <code>try/except ValueError</code> + rango
<code>read_float(msg)</code>	Pide decimal validado	Sí — <code>try/except ValueError</code>
<code>parse_order_line(line)</code>	Procesa “cant nombre” → tupla	Sí — <code>try</code> + <code>split/strip</code>
<code>add_order(order_list)</code>	Agrega pedido con datos	Sí — todo validado
<code>list_orders(order_list)</code>	Muestra todos los pedidos	No — solo recorre
<code>search_by_phone(order_list, phone)</code>	Busca pedido por teléfono	Sí — limpia teléfono con <code>replace</code>
<code>show_total_sales(order_list)</code>	Suma y muestra ventas totales	No — solo suma
<code>show_menu()</code>	Muestra opciones	No

Import permitidos: `import random` (para número de pedido), `import math` (opcional).

Sin persistencia: toda la información vive en una lista de diccionarios (o de tuplas) en memoria. Al cerrar el programa, los datos se pierden.

Esquema de un pedido (en memoria):

```
# Cada pedido es un diccionario:
pedido = {
    "customer": "María",
    "phone": "1123456789",
    "items": [("muzzza", 2, 2400.0), ("napo", 1, 1400.0)],
    "total": 3800.0,
    "ticket": 457
}
```

4. Práctica guiada (35 min)

Escenario: empezamos a construir el programa base del TP-U3 con las funciones de gestión de pedidos.

Código completo (`pedidos.py`):

```
import random

def read_int(message, min_val=None, max_val=None):
    """Pide un entero validado con rango opcional."""
    while True:
```

```

    try:
        value = int(input(message))
        if min_val is not None and value < min_val:
            print(f"⚠ El mínimo es {min_val}; intentá de nuevo.")
            continue
        if max_val is not None and value > max_val:
            print(f"⚠ El máximo es {max_val}; intentá de nuevo.")
            continue
        return value
    except ValueError:
        print("⚠ Eso no es un número entero; intentá de nuevo.")

def read_float(message):
    """Pide un decimal validado."""
    while True:
        try:
            return float(input(message))
        except ValueError:
            print("⚠ Usá punto decimal (ej: 12.50); intentá de nuevo.")

def clean_phone(phone_text):
    """Saca espacios y guiones del teléfono, deja solo dígitos."""
    return phone_text.replace(" ", "").replace("-", "")

def parse_order_line(line):
    """Procesa '2 muzza' y devuelve (cantidad, nombre)."""
    parts = line.strip().split()
    if len(parts) < 2:
        print("⚠ Formato: cantidad nombre (ej: 2 muzza).")
        return None
    try:
        quantity = int(parts[0])
        name = " ".join(parts[1:])
        return (quantity, name)
    except ValueError:
        print("⚠ La cantidad debe ser un número entero.")
        return None

def add_order(order_list):
    """Agrega un pedido completo al sistema."""
    print("\n--- Nuevo pedido ---")
    customer = input("Nombre del comprador: ").strip()
    phone_raw = input("Teléfono: ").strip()
    phone = clean_phone(phone_raw)

    # Pide las pizzas del pedido
    items = []
    while True:
        line = input("Pizza (cant nombre) o ENTER para terminar: ").strip()
        if line == "":
            break

```

```

    parsed = parse_order_line(line)
    if parsed:
        qty, name = parsed
        # Precio base
        price = read_float(f" Precio de {name}: $")
        subtotal = qty * price
        items.append((name, qty, price, subtotal))
        print(f" ✓ {qty}x {name} agregada.")

    if not items:
        print("❌ El pedido debe tener al menos una pizza.")
        return

    # Calcula total
    total = sum(sub for _, _, sub in items)
    ticket = random.randint(1000, 9999)
    order_list.append({
        "customer": customer,
        "phone": phone,
        "items": items,
        "total": total,
        "ticket": ticket
    })
    print(f"✓ Pedido registrado. Ticket #{ticket} – Total: ${total:.2f}")

def list_orders(order_list):
    """Muestra todos Los pedidos registrados."""
    if not order_list:
        print("\n❌ No hay pedidos registrados.")
        return
    for order in order_list:
        print(f"\nTicket #{order['ticket']} – {order['customer']}")
        print(f" Tel: {order['phone']}")
        for name, qty, price, sub in order["items"]:
            print(f" {qty}x {name}: ${price:.2f} → ${sub:.2f}")
        print(f" Total: ${order['total']:.2f}")

def search_by_phone(order_list, search_phone):
    """Busca pedidos por teléfono (búsqueda parcial)."""
    found = []
    for order in order_list:
        if search_phone in order["phone"]:
            found.append(order)
    return found

def show_menu():
    print("\n" + "=" * 35)
    print(" 🍕 Pizzería – Sistema de Pedidos")
    print("=" * 35)
    print("1. Agregar pedido")
    print("2. Listar pedidos")
    print("3. Buscar por teléfono")
    print("4. Salir")

```

```

print("\n4. Mostrar ventas totales")
print("\n0. Salir")
return input("Elegí una opción: ").strip()

if __name__ == "__main__":
    orders = []
    print("=== Sistema de Pedidos de la Pizzería ===")

    while True:
        option = show_menu()

        if option == "0":
            print("\n ¡Hasta luego!")
            break
        elif option == "1":
            add_order(orders)
        elif option == "2":
            list_orders(orders)
        elif option == "3":
            search = input("Teléfono a buscar: ").strip()
            results = search_by_phone(orders, clean_phone(search))
            if results:
                print(f"\nSe encontraron {len(results)} pedido(s):")
                for order in results:
                    print(f"  #{order['ticket']} - {order['customer']} - ${order['total']:.2f}")
            else:
                print("\n No se encontraron pedidos con ese teléfono.")
        elif option == "4":
            if not orders:
                print("\n No hay pedidos registrados.")
            else:
                total = sum(o["total"] for o in orders)
                count = len(orders)
                print(f"\n Ventas totales: ${total:.2f} ({count} pedido(s))")
        else:
            print("\n Opción inválida. Elegí 0-4.")

```

Salida esperada (secuencia de prueba mínima):

```

=== Sistema de Pedidos de la Pizzería ===

=====
  🍕 Pizzería – Sistema de Pedidos
=====

1. Agregar pedido
2. Listar pedidos
3. Buscar por teléfono
4. Mostrar ventas totales
0. Salir
Elegí una opción: 1

--- Nuevo pedido ---
Nombre del comprador: María
Teléfono: 11 2345 6789
Pizza (cant nombre) o ENTER para terminar: 2 muzza
  Precio de muzza: $1200
  ✓ 2x muzza agregada.
Pizza (cant nombre) o ENTER para terminar:
✓ Pedido registrado. Ticket #3142 – Total: $2400.00

Elegí una opción: 4

🍕 Ventas totales: $2400.00 (1 pedido(s))

```

Pasos: 1. Escribí `pedidos.py` completo. 2. Ejecutalo y probá agregar al menos 2 pedidos. 3. Probá la búsqueda por teléfono. 4. Probá opciones inválidas.

5. Ejercicio independiente (25 min)

Consigna: agregá una **opción 5. Ver detalle de pedido** que pida un número de ticket y muestre el pedido completo de ese ticket. Si el ticket no existe, mostrá un mensaje.

Además, modificá la opción 3 (búsqueda por teléfono) para que, si se encuentra un solo pedido, muestre automáticamente el detalle completo (no solo el resumen).

Pista: recorré la lista de pedidos comparando `order["ticket"]` con el número ingresado. Usá `read_int` para leer el número de ticket.

Solución: en el anexo docente.

6. Extensión y consolidación (20 min)

Para quienes terminan el ejercicio base:

- Agregar opción 6. Pedido más caro:** mostrá el pedido con el total más alto (usá `max()` con `key=lambda` o un `for` manual).
- Validar precio positivo:** que `read_float` no acepte precios negativos.
- Confirmación al agregar:** antes de guardar el pedido, mostrá un resumen y preguntá “¿Confirmar? (s/n)”.

7. Cierre (10 min)

Qué te llevás

- El TP-U3 ya tiene esqueleto: funciones de validación, menú, colecciones en memoria.
- Cada pedido se modela como un diccionario: es fácil de entender y de recorrer.
- La búsqueda con `in` en cadenas permite encontrar coincidencias parciales.
- El próximo encuentro es la entrega: vamos a cerrar el TP, hacer commits y push.

Lo que viene

En el **Encuentro 25** hacemos el **cierre de la Unidad 3**: repaso general, trabajo en el TP y **entrega** con carpeta nueva, commits y push al repositorio.

8. Errores comunes y trampas

Error	Causa	Fix
El diccionario del pedido tiene claves mal escritas	Typo en <code>"customer"</code> vs <code>"costumer"</code>	Usar el mismo nombre en todas partes; copiar del ejemplo
No actualiza la lista después de agregar	<code>add_order</code> recibe la lista pero no la modifica	<code>order_list.append(...)</code> modifica la lista in-place
La búsqueda por teléfono nunca encuentra nada	No aplica <code>clean_phone</code> al buscar	Aplicar <code>clean_phone</code> tanto al guardar como al buscar
<code>sum()</code> sobre la lista vacía da 0	No es error, pero confunde si no hay pedidos	Chequear <code>if not orders:</code> antes de mostrar ventas
El menú no reconoce opciones con espacios	<code>input()</code> sin <code>.strip()</code>	Siempre hacer <code>.strip()</code> sobre el resultado de <code>input()</code>
Olvida <code>import random</code> al principio del archivo	<code>random.randint</code> no está disponible	<code>import random</code> debe estar antes de cualquier función que lo use
Usa <code>break</code> para salir del <code>while</code> de pizzas pero después no guarda el pedido	Corta antes de calcular total y ticket	Verificar el flujo: <code>while</code> solo para las pizzas, guardar después
El precio de la pizza se pide con <code>input()</code> sin validar	No usa <code>read_float</code> para el precio	Reemplazar <code>input()</code> por <code>read_float()</code>

Encuentro 25 — Cierre U3: repaso y TP

Procesamiento de texto y validación · Encuentro de cierre de unidad

1. Metadatos de bloque

Campo	Detalle
Encuentro	25 de 36
Unidad	3 — Procesamiento de texto y validación
Eje temático	3 — Procesamiento de texto y validación
Carácter/Objetivo	Actitudinal
Estructura	cierre
Duración teórica	120 minutos (2 horas reloj)
TP obligatorio	TP-U3: menú de consola validado
Concepto nuevo	Cierre U3: repaso y TP
Requisitos previos	Toda la Unidad 3: métodos de texto, validación con <code>try/except</code> , módulos, menú en memoria
Uso de celular	No permitido
Organización del trabajo	Grupos de 2 personas. Hoy se entrega el TP-U3. La primera mitad es repaso guiado; la segunda es trabajo autónomo y entrega.

Reparto de tiempos (encuentro de cierre)

Momento	Tiempo
Apertura	10 min
Consolidación	40 min
Trabajo del TP	45 min
Ciclo de entrega	15 min
Cierre	10 min
Total	120 min

2. Objetivos de aprendizaje

1. Sistematizar los conceptos de la Unidad 3: métodos de texto, validación, módulos y menú en memoria.
2. Finalizar el TP-U3 agregando las funcionalidades faltantes y verificando su correcto funcionamiento.
3. Crear la carpeta `tp-u3/` con el programa, realizar commit y push al repositorio grupal.
4. Reflexionar sobre el proceso de integración: cómo se combinan funciones, colecciones, texto y validación en un programa útil.

3. Apertura (10 min)

Charla rápida: la pizza completa

Armar una pizza es juntar ingredientes que vienen de distintos lados: la masa (funciones), la salsa (validación), el queso (colecciones), los toppings (métodos de texto). Solos no son una pizza; juntos, sí. La Unidad 3 fue eso: tomar las piezas que ya conocíamos (funciones, colecciones, `if`, `while`) y agregarles el procesamiento de texto y la validación para que el programa no se rompa ante cualquier cosa que tipee el usuario. Hoy cerramos esa pizza y la entregamos.

Repaso rápido de 5 minutos

Concepto clave	Lo que hace	Ejemplo mínimo
<code>split()</code> , <code>strip()</code> , <code>join()</code> , <code>replace()</code>	Procesan texto del usuario	<code>" a, b ".split(",")</code> → <code>[" a", " b "]</code>
<code>try/except ValueError</code>	Atrapa error de conversión	<code>int("hola")</code> → <code>ValueError</code>
Patrón de reintento	<code>while True</code> + <code>try/except</code> + <code>return</code>	Ciclo hasta obtener dato válido
<code>import random</code>	Números aleatorios	<code>random.randint(1, 100)</code>
Menú en memoria	<code>while True</code> + opciones + <code>break</code>	Las opciones son strings, se comparan con <code>==</code>
Sin persistencia	Todo vive en memoria	Al cerrar, se pierde; en la próxima ejecución arranca vacío

4. Consolidación (40 min)

Ejercicio de sistematización: armado guiado del TP-U3

Consigna del TP-U3: > Desarrollá un programa de gestión de pedidos para una pizzería. El programa debe: > 1. Registrar pedidos con nombre del comprador, teléfono, lista de pizzas (cantidad + nombre + precio unitario) y número de ticket. > 2. Listar todos los pedidos registrados. > 3. Buscar pedidos por teléfono. > 4. Mostrar las ventas totales. > 5. Usar validación `try/except ValueError` para toda entrada numérica. > 6. Usar métodos de texto (`split`, `strip`, `join`, `replace`) para procesar la entrada. > 7. Generar número de ticket con `random.randint`. > 8. Ser un solo archivo `pedidos.py` en la carpeta `tp-u3/`.

Checklist de verificación del TP-U3 (para que cada grupo lo revise)

- ☐ El programa arranca y muestra el menú
- ☐ Opción 1: Agregar pedido — pide nombre (texto), teléfono (texto, se limpia), pizzas (cant + nombre + precio)
- ☐ La entrada de cantidades y precios valida con `try/except ValueError` y mensaje accionable
- ☐ Opción 2: Listar pedidos — muestra todos los registrados
- ☐ Opción 3: Buscar por teléfono — funciona con o sin espacios/guiones
- ☐ Opción 4: Mostrar ventas totales
- ☐ Opción 0: Salir — corta el programa
- ☐ Opciones inválidas → mensaje “Opción inválida”
- ☐ El código tiene funciones arriba y ejecución en `if __name__ == "__main__":`
- ☐ Sin `except:` desnudo
- ☐ Sin `open()`, archivos, CSV ni JSON
- ☐ Sin clases, lambda ni comprehensions avanzadas

Si un grupo ya completó todo en la clase 24

Que trabaje en funcionalidades extra para su programa: - Opción “Modificar pedido”: buscar por ticket, cambiar cantidad o precio. - Opción “Pedido más caro”: mostrar el pedido con mayor total. - Ordenar pedidos por total (de mayor a menor) con

`sorted()` y mostrarlos.

5. Trabajo del TP (45 min)

Trabajo autónomo en grupos

Cada grupo trabaja en su programa: - Los que terminaron: pulen detalles, agregan funcionalidades extra, verifican el checklist. - Los que no terminaron: completan lo que falta con ayuda del docente. - El docente circula, resuelve dudas puntuales, verifica avances.

Pautas para la entrega

1. **Crear la carpeta del TP:** dentro del repositorio grupal, crear `tp-u3/` (si no existe).
2. **Copiar el archivo:** `pedidos.py` adentro de `tp-u3/`.
3. **Verificar ejecución:** `python pedidos.py` desde la terminal, en la carpeta `tp-u3/`.
4. **Probar casos borde:**
 - Entrada no numérica en cantidad → debe pedir de nuevo
 - Teléfono con espacios → debe limpiarlos
 - Pedido sin items → debe rechazar
 - Opción 9 (inexistente) → "Opción inválida"
 - Buscar teléfono que no existe → "No se encontraron pedidos"

6. Ciclo de entrega (15 min)

Pasos de entrega (lo hace cada grupo, guiado por el docente)

```
# 1. Verificar que estamos en el repositorio correcto
git status

# 2. Agregar la carpeta tp-u3/ y el archivo
git add tp-u3/pedidos.py

# 3. Commit con mensaje descriptivo
git commit -m "tp-u3: menú de consola validado para pizzería"

# 4. Push al remoto (GitHub)
git push
```

Nota importante: si el repositorio usa mono-rama `main`, se pushea directo. No crear ramas hasta la Unidad 4.

Registro de entrega

El docente registra qué grupos completaron la entrega. Los grupos que no terminan pueden hacerlo en la semana y pushear antes de la clase siguiente. Informar por el medio habitual del curso (grupo de WhatsApp / aula virtual).

7. Cierre (10 min)

Qué te llevás

- La Unidad 3 nos dio las herramientas para que los programas **no se rompan** con la entrada del usuario.

- El TP-U3 muestra cómo integrar todo: funciones, colecciones, texto y validación.
- Git no es solo para código: es la bitácora del avance. Cada commit documenta un paso.
- Sin persistencia: los datos viven en memoria mientras el programa corre. No se necesita archivos para tener un programa útil.
- Próximo paso: la evaluación de la Unidad 3.

Lo que viene

En el **Encuentro 26** vamos a hacer la **Evaluación de la Unidad 3**: un ejercicio integrador similar al TP, pero individual, para verificar lo que aprendimos.

8. Errores comunes y trampas

Error	Causa	Fix
El programa no guarda los pedidos entre ejecuciones	Sin persistencia es esperado: el programa arranca vacío cada vez	No es un bug; explicar que la persistencia se ve más adelante
Olvida hacer <code>git add</code> antes del commit	<code>git commit</code> solo commitea lo que está en el stage	<code>git status</code> para ver qué falta agregar
El commit tiene mensaje genérico como "cambios"	No documenta qué se hizo	Usar el formato del curso: <code>"tp-u3: <avance>"</code>
<code>git push</code> falla porque el remoto no está configurado	Es la primera vez que pushean desde esa máquina	Verificar con <code>git remote -v</code> y configurar si falta
El programa tiene errores que no se detectaron en clase	No probaron los casos borde	Usar la checklist de verificación antes de entregar
La carpeta se llama <code>TP-U3/</code> o <code>Tp3/</code> en vez de <code>tp-u3/</code>	Inconsistencia con la nomenclatura del curso	El nombre debe ser <code>tp-u3/</code> (minúsculas, guión) para mantener consistencia con las unidades anteriores
Olvida el <code>if __name__ == "__main__":</code>	El código suelto se ejecuta al importar	Revisar la convención del curso: funciones arriba, ejecución en el bloque principal

Evaluación de la Unidad 3 — Encuentro 26

Evaluación de la instancia «Evaluación de la Unidad 3 — Encuentro 26» · Curso: Programación en Python. Documento de **metadatos y acuerdos de la instancia**, en registro docente formal. El material del alumno es la versión `evaluacion-u3-version-a` y sus versiones equivalentes generadas; sus soluciones y criterios de corrección van en el anexo docente separado (`evaluacion-u3-version-a-anexo-docente.md`).

1. Identificación

Campo	Detalle
Instancia	Evaluación de la Unidad 3 — Encuentro 26
Unidad evaluada	3 — Procesamiento de texto y validación
Eje temático	3 — Procesamiento de texto y validación
Carácter/Objetivo	Evaluación práctica individual: verificar la capacidad de procesar texto con métodos de cadenas, validar entrada con excepciones y construir un menú interactivo robusto.
Destinatarios	Todo el curso
Duración teórica	120 minutos (2 horas reloj)
Uso de celular	No permitido
Documentos de la instancia	<code>evaluacion-u3.md</code> · <code>evaluacion-u3-version-a.md</code> · <code>evaluacion-u3-version-a-anexo-docente.md</code> (más las versiones equivalentes B generadas desde la A)

2. Estructura del encuentro (120 min)

Momento	Tiempo	Qué ocurre
Apertura e instrucciones	10 min	El docente explica la dinámica: defensa del TP-U3 + prueba individual. Asigna versiones.
Defensa individual del TP (grupos)	30 min	Rotación de grupos para defensa del TP-U3.
Prueba práctica individual	70 min	Cada estudiante resuelve su ejercicio.
Cierre y recogida	10 min	Entrega por Git. Devolución anunciada para el encuentro siguiente.
Total	120 min	

3. Regla canónica de la instancia

La entrega del TP-U3 se verificó en el encuentro 25. En este encuentro 26 se realiza la defensa y la prueba individual. Devolución al inicio del encuentro 27.

4. Defensa individual del TP (modalidad)

Por grupo: demo del TP-U3, revisión del repositorio, pregunta individual a cada integrante sobre validación, métodos de texto o

menú.

5. Alcance

Incluido: métodos de cadenas (`split`, `strip`, `join`, `replace`), `try/except ValueError` con reintento, menú en memoria, funciones, colecciones, import puntual de `random/math`.

Excluido: archivos, POO, lambda, comprehensions avanzadas.

Objetivo mínimo: procesar texto de entrada con métodos de cadenas, validar con excepciones y construir un menú interactivo que no se rompa ante ninguna entrada.

6. Prueba práctica individual

Un único ejercicio en un archivo `.py`. Puntaje total: 100 puntos.

- **Parte 1 — Parseo de texto (30 pts):** procesar una línea de texto con `split`, `strip`, validar formato.
- **Parte 2 — Validación y registro (40 pts):** función que valide datos numéricos y de texto, con `try/except` y reintento.
- **Parte 3 — Menú y reporte (20 pts):** menú que use las funciones anteriores y muestre un listado.
- **Parte 4 — Ítems conceptuales (10 pts).**

7. Criterios de calificación

Componente	Puntaje
Estructura y convenciones	10 pts
Parte 1 — Parseo	30 pts
Parte 2 — Validación	40 pts
Parte 3 — Menú y reporte	10 pts
Ítems conceptuales	10 pts

8. Condiciones de resolución

Individual, apuntes propios, sin celular. Convenciones del curso obligatorias. Entrega por Git en `evaluacion-u3/`.

9. Regla de equivalencia entre versiones

Misma estructura y requisitos. Cambia el dominio (pizzería ↔ veterinaria). Misma tabla de puntaje.

10. Asignación de versiones

Fila impar → A (pizzería). Fila par → B (veterinaria).

11. Devolución

Encuentro 27. Recuperación al inicio del encuentro 28 si menos de 60 pts.

Evaluación de la Unidad 3 — Encuentro 26 — Versión A

Dominio de esta versión: **pizzería** (gestión de pedidos). Duración: 70 minutos. Puntaje total: 100 puntos. Resolución individual, con computadora, sin celular.

Antes de empezar

- Carpeta `evaluacion-u3/`, archivo `pizzeria.py`.
- Seguí las convenciones del curso. Toda entrada numérica validada con `try/except ValueError`.
- Métodos de texto: usá `split`, `strip`, `join`, `replace` donde corresponda.
- Al terminar: commit con `"evaluacion-u3: entrega"` y push.

Objetivos de la prueba

1. Procesar texto del usuario con métodos de cadenas.
2. Validar entrada con `try/except` y reintento.
3. Construir un menú interactivo con opciones que procesen datos.

Material provisto

No hay esqueleto. Datos de ejemplo: una lista `orderList` vacía que se llena durante la ejecución.

Parte 1 — Parsear un pedido (30 puntos)

El usuario ingresa una línea de texto con formato: `nombre del comprador, precioUnitario, cantidad`. Por ejemplo: `"Ana Lopez, 12.5, 3"`.

Ítem	Consigna	Pts
1a	Función <code>parse_order(order_text)</code> : recibe un string, usa <code>split(",")</code> y <code>strip()</code> para separar y limpiar los campos. Devuelve un dict con <code>customerName</code> , <code>orderPrice</code> (float) y <code>quantity</code> (int).	15
1b	Si la línea no tiene exactamente 3 campos separados por coma, mostrar "Formato inválido. Usá: nombre, precio, cantidad."	10
1c	Si <code>orderPrice</code> o <code>quantity</code> no son numéricos, lanzar <code>ValueError</code> y capturarlo con mensaje claro.	5
Total Parte 1		30

Parte 2 — Validación y registro (40 puntos)

Ítem	Consigna	Pts
------	----------	-----

Ítem	Consigna	Pts
2a	Función <code>add_order()</code> : pide al usuario una línea de texto, llama a <code>parse_order()</code> , y si es válida agrega el pedido a <code>orderList</code> . Usá <code>try/except ValueError</code> para capturar errores de conversión.	15
2b	Función <code>valid_phone(phone)</code> : recibe un string, limpia espacios y guiones con <code>replace()</code> y <code>strip()</code> , verifica que tenga al menos 7 dígitos. Devuelve <code>True</code> o <code>False</code> .	10
2c	Al agregar un pedido, pedir también teléfono y validarlo con <code>valid_phone()</code> . Si no es válido, pedir de nuevo.	10
2d	Generar número de ticket aleatorio con <code>random.randint(1000, 9999)</code> y mostrarlo al confirmar el pedido. Hacé <code>import random</code> al inicio.	5
Total Parte 2		40

Parte 3 — Menú y reporte (20 puntos)

Ítem	Consigna	Pts
3a	Menú principal: 1-Agregar pedido, 2-Listar pedidos, 3-Salir	5
3b	Opción 2 (Listar): mostrar todos los pedidos con nombre, teléfono, precio, cantidad y ticket. Formato de precio con 2 decimales.	10
3c	Opción inválida → “Opción inválida.”	5
Total Parte 3		20

Parte 4 — Ítems conceptuales (10 puntos)

Ítem	Pregunta	Pts
4a	¿Qué hace <code>split(",")</code> si el usuario no pone ninguna coma en el texto?	5
4b	¿Por qué conviene usar <code>try/except ValueError</code> en lugar de <code>if</code> para verificar que un texto sea numérico?	5

Batería de verificación

Prueba	Pedido	Salida esperada
Opción 1 (válido)	“Ana, 12.5, 3”	Pedido agregado, muestra ticket aleatorio
Opción 1 (mal formato)	“Ana, 12.5”	“Formato inválido. Usá: nombre, precio, cantidad.”

Prueba	Pedido	Salida esperada
Opción 1 (precio no numérico)	"Ana, doce, 3"	Mensaje de error de conversión
Opción 1 (teléfono inválido)	"Ana, 12.5, 3", tel: "abc"	Pide teléfono de nuevo
Opción 2	Listar	Muestra todos los pedidos registrados
Opción 3	Salir	"Saludos!"

Al terminar

Verificá ejecución: `python pizzeria.py`. Commit y push.

Evaluación de la Unidad 3 — Encuentro 26 — Versión A

Dominio de esta versión: **veterinaria** (gestión de pedidos). Duración: 70 minutos. Puntaje total: 100 puntos. Resolución individual, con computadora, sin celular.

Antes de empezar

- Carpeta `evaluacion-u3/`, archivo `pizzeria.py`.
- Seguí las convenciones del curso. Toda entrada numérica validada con `try/except ValueError`.
- Métodos de texto: usá `split`, `strip`, `join`, `replace` donde corresponda.
- Al terminar: commit con `"evaluacion-u3: entrega"` y push.

Objetivos de la prueba

1. Procesar texto del usuario con métodos de cadenas.
2. Validar entrada con `try/except` y reintento.
3. Construir un menú interactivo con opciones que procesen datos.

Material provisto

No hay esqueleto. Datos de ejemplo: una lista `appointmentList` vacía que se llena durante la ejecución.

Parte 1 — Parsear un turno (30 puntos)

El usuario ingresa una línea de texto con formato: `nombre del mascota, precioUnitario, cantidad`. Por ejemplo: `"Ana Lopez, 12.5, 3"`.

Ítem	Consigna	Pts
1a	Función <code>parse_order(order_text)</code> : recibe un string, usa <code>split(",")</code> y <code>strip()</code> para separar y limpiar los campos. Devuelve un dict con <code>petName</code> , <code>consultPrice</code> (float) y <code>quantity</code> (int).	15
1b	Si la línea no tiene exactamente 3 campos separados por coma, mostrar "Formato inválido. Usá: nombre, tarifa, cantidad."	10
1c	Si <code>consultPrice</code> o <code>quantity</code> no son numéricos, lanzar <code>ValueError</code> y capturarlo con mensaje claro.	5
Total Parte 1		30

Parte 2 — Validación y registro (40 puntos)

Ítem	Consigna	Pts
------	----------	-----

Ítem	Consigna	Pts
2a	Función <code>add_order()</code> : pide al usuario una línea de texto, llama a <code>parse_order()</code> , y si es válida agrega el turno a <code>appointmentList</code> . Usá <code>try/except ValueError</code> para capturar errores de conversión.	15
2b	Función <code>valid_phone(phone)</code> : recibe un string, limpia espacios y guiones con <code>replace()</code> y <code>strip()</code> , verifica que tenga al menos 7 dígitos. Devuelve <code>True</code> o <code>False</code> .	10
2c	Al agregar un turno, pedir también teléfono y validarlo con <code>valid_phone()</code> . Si no es válido, pedir de nuevo.	10
2d	Generar número de ticket aleatorio con <code>random.randint(1000, 9999)</code> y mostrarlo al confirmar el turno. Hacé <code>import random</code> al inicio.	5
Total Parte 2		40

Parte 3 — Menú y reporte (20 puntos)

Ítem	Consigna	Pts
3a	Menú principal: 1-Agregar turno, 2-Listar pedidos, 3-Salir	5
3b	Opción 2 (Listar): mostrar todos los pedidos con nombre, teléfono, tarifa, cantidad y ticket. Formato de tarifa con 2 decimales.	10
3c	Opción inválida → “Opción inválida.”	5
Total Parte 3		20

Parte 4 — Ítems conceptuales (10 puntos)

Ítem	Pregunta	Pts
4a	¿Qué hace <code>split(",")</code> si el usuario no pone ninguna coma en el texto?	5
4b	¿Por qué conviene usar <code>try/except ValueError</code> en lugar de <code>if</code> para verificar que un texto sea numérico?	5

Batería de verificación

Prueba	Turno	Salida esperada
Opción 1 (válido)	“Ana, 12.5, 3”	Turno agregado, muestra ticket aleatorio

Prueba	Turno	Salida esperada
Opción 1 (mal formato)	"Ana, 12.5"	"Formato inválido. Usá: nombre, tarifa, cantidad."
Opción 1 (tarifa no numérico)	"Ana, doce, 3"	Mensaje de error de conversión
Opción 1 (teléfono inválido)	"Ana, 12.5, 3", tel: "abc"	Pide teléfono de nuevo
Opción 2	Listar	Muestra todos los pedidos registrados
Opción 3	Salir	"Saludos!"

Al terminar

Verificá ejecución: `python pizzeria.py`. Commit y push.